

Scorm: Naming, Scope and Referencing environment

Teacher: Dr. Nguyen Hua Phung

Alias Vs. Polymorphism

- **Alias (bí danh) tức là một vùng nhớ có nhiều tên cùng trỏ về.** Tương tự như một người có nhiều tên. Ví dụ: `int &a = p` → `a` là 1 tham chiếu cũng là 1 bí danh của `p`
- **Polymorphism (đa hình) tức là cùng một tên nhưng nhiều đối tượng/vùng nhớ khác nhau sử dụng.** Diễn hình của đa hình là overriding và overloading.
 - overriding (ghi đè) tức là viết đè một phương thức/thuộc tính mới lên một phương thức/thuộc tính có sẵn, với các tham số đầu vào giống hệt. Cho phép thay đổi hành vi ở runtime.
 - overloading là cùng 1 tên nhưng khác kiểu dữ liệu, lượng tham số đầu vào....

Name Binding

- **Thời gian thiết kế/định nghĩa ngôn ngữ (language design time):** thực hiện cho việc định nghĩa tên của kiểu dữ liệu. Ví dụ, ta có kiểu nguyên, ta mặc định nếu người ta dùng kiểu nguyên thì phải ghi chữ `int`, hoặc phép cộng là dấu `+`. Các độ ưu tiên và đệ quy (associativity and precedence) cũng được thực hiện khi thiết kế
- **Thời gian hiện thực ngôn ngữ (language implementation time):** trong lúc làm design không nêu ra mà lúc làm chương trình dịch cần phải quyết định. Tiêu biểu: sử dụng kiểu nguyên thì cần cấp cho bao nhiêu byte? 4 byte, 16 byte ...
 - **Lưu ý, nếu lúc thiết kế không nói thì lúc làm chương trình dịch mới phải quyết định.** Ví dụ như lúc thiết kế không nói là `int` phải cấp phát 4 byte thì lúc làm chương trình dịch (implementation time) mới làm. còn nếu đã định nghĩa luôn lúc thiết kế (design time) thì không được làm lúc dịch.
- **Thời gian lập trình (programming time)** là ràng buộc do người lập trình quyết định (ví dụ: ta mong muốn `x` là kiểu nguyên → cần khai báo `int x...`), ngoài ra còn đặt tên biến là gì ... → Người dùng định nghĩa như thế nào thì đều là ràng buộc ở programming time.
- **Thời gian dịch (compilation time):** là thời gian xảy ra ràng buộc giữa lệnh gọi và chương trình con được gọi. Ví dụ: gọi hàm.
- **Thời gian nối (linking time):** thời gian xảy ra các ràng buộc nối các file với nhau (dùng khi chia file). Ví dụ: **địa chỉ tương đối.**
- **Thời gian nạp (load time):** thời gian nạp từ đĩa cứng (ổ C, D, E ...) lên bộ nhớ trong (RAM/ROM) để chạy chương trình. ví dụ, **địa chỉ tuyệt đối.**
- **Thời gian chạy (run time):** Lúc thực thi mới chạy, ví dụ như nhập giá trị để thực hiện chương trình sau khi chạy.

Update:

- Định nghĩa ngôn ngữ: lúc viết đặc tả (specification)
- Hiện thực ngôn ngữ: Lúc viết OPLang (Assignment 1,2,3,4)
- Thời gian lập trình: Người khác sử dụng ngôn ngữ để viết một đoạn mã
- Thời gian dịch: bỏ đoạn mã vào để dịch (compile)
- Bấm chương trình để chạy → thực thi

Dangling reference

- **Dangling reference (Tham chiếu treo) xảy ra khi ràng buộc vẫn còn trong khi đối tượng đã biến mất.**
- Ví dụ cho đoạn code sau:

```
int x;  
int* p, *q;  
p = new int(x); // memory allocation  
q = p;          // copy reference  
delete p;  
*q;             // dangling reference
```

- Trình tự thực hiện của đoạn code:
 - Tạo 2 biến p và q trở tới kiểu int, biến x có kiểu int.
 - p trở tới một đối tượng được khởi tạo có giá trị là x. lúc này có 3 đối tượng: p, q, new int(x). Điều này tạo ra ràng buộc giữa p và đối tượng mang giá trị khởi tạo x.
 - q = p copy giá trị của p cho q, tức q cũng sẽ trở tới đối tượng mang giá trị khởi tạo x.
 - delete p xóa đối tượng new int(x) thông qua con trỏ p, xóa ràng buộc giữa p và đối tượng mang giá trị khởi tạo x.
 - *q → vẫn còn ràng buộc giữa q và đối tượng mang giá trị khởi tạo x, trong khi đối tượng này đã bị hủy bởi lệnh delete dẫn tới lỗi tham chiếu treo (dangling reference).
- Lỗi này cực kì nghiêm trọng vì khi thực hiện xóa đối tượng, vùng nhớ này sẽ được máy cấp phát để thực hiện một chương trình khác, lúc này ràng buộc treo sẽ làm sai lệch hoàn toàn chương trình. Vì vậy, các ngôn ngữ bậc cao thường có biện pháp để ngăn lỗi này xảy ra.

Memory Leak

- Là lỗi xảy ra khi xóa tham chiếu tới các đối tượng, khiến cho không còn phương án nào để truy xuất đối tượng nữa. Lúc này, đối tượng sẽ trở thành rác (garbage) gây chiếm dụng bộ nhớ.
- Ví dụ cho đoạn code sau:

```
int x;  
int *p = new int(x);  
p = null;    // memory leak
```

- Trong đoạn code trên, sau khi khởi tạo con trỏ p trỏ đến đối tượng được khởi tạo với giá trị x, ta thay đổi con trỏ p trỏ tới null, tức xóa ràng buộc giữa p và đối tượng kia.
- Khi này, không có cách nào để tham chiếu tới đối tượng x nữa, đối tượng trở thành rác và xảy ra hiện tượng memory leak.

Object Allocation

- Có 4 loại cấp phát bộ nhớ thông dụng:
 - **Cấp phát tĩnh (static):** Thời gian sống bằng với thời gian thực thi toàn bộ chương trình (the whole program). Thường là những biến khai báo ngoài các hàm hoặc khai báo với từ khóa `static`.
 - **Cấp phát trên stack (Stack Dynamic):** cấp phát theo nguyên tắc LIFO. Ví dụ điển hình là các biến cục bộ (local variable), với thời gian sống bằng thời gian sống của hàm (từ lúc gọi cho tới lúc kết thúc hàm).
 - **Cấp phát rõ ràng trên heap (Explicit Heap Dynamic):** cấp phát rõ ràng bằng các lệnh, ví dụ như trong ngôn ngữ C/C++ là `new`, `malloc`, `calloc`. Các đối tượng được khởi tạo rõ ràng, có thời gian sống lâu hơn thời gian sống của hàm và kết thúc trước thời gian sống của toàn bộ chương trình.
 - **Cấp phát ngầm định (Implicit Heap Dynamic):** Tương tự, nhưng là những cấp phát được hiểu ngầm định

Block Scope

- Tầm vực (Scope) là miền văn bản mà khai báo có hiệu lực. Có 2 loại tầm vực: tầm vực tĩnh và tầm vực động.
- Tầm vực tĩnh bao gồm các tính chất:
 - Một tham chiếu đến định danh (biến, hàm, ...) luôn được liên kết với khai báo gần nhất trong phạm vi hiện tại.
 - Một khai báo bên trong block sẽ không thể thấy được bên ngoài block.
 - Các khai báo trong **khối bao ngoài** sẽ **có thể thấy được** trong các khối bên trong, **trừ khi** chúng bị **khai báo lại**. Điều này có nghĩa nếu bạn khai báo lại một biến đề lên biến ở ngoài block, biến ở ngoài block sẽ bị mất hiệu lực hoạt động.
 - **Các khối có thể được đặt tên (đặc biệt trong một số ngôn ngữ) và tên đó được coi là khai báo cục bộ của khối ngoài.**

- Tầm vực động chỉ được biết tầm hoạt động khi thực thi chương trình (runtime).

Quiz

1. Cho phát biểu C++ sau:

```
x = x + y * 10;
```

Trong biểu thức bên vế phải, phép nhân * được tính toán trước phép +; ràng buộc này xảy ra vào thời gian nào?

- Thời gian dịch (Compiling time)
- Hiện thực ngôn ngữ (Language implementation time)
- Thời gian thực thi (Runtime)
- Định nghĩa ngôn ngữ (Language design time)**

2. Cho phát biểu Python sau:

```
x = x * y ** 3;
```

Trong biểu thức bên vế phải, phép nhân * được tính toán sau phép **; ràng buộc này xảy ra vào thời gian nào?

- Thời gian dịch (Compiling time)
- Định nghĩa ngôn ngữ (Language design time)**
- Thời gian thực thi (Runtime)
- Hiện thực ngôn ngữ (Language implementation time)

3. Cho biết trong đoạn mã C++ sau, đối tượng nào có nhiều tên và những tên này là gì?

```
int *p = new int;
```

```
int *q = p;
```

- Đối tượng có nhiều tên là đối tượng được tạo ra bởi lệnh "new int" và các tên của đối tượng này là p và q
- Đối tượng có nhiều tên là p và các tên của nó là p và q
- Đối tượng có nhiều tên là đối tượng được tạo ra bởi lệnh "new int" và các tên của đối tượng này là *p và *q**
- Không có câu nào đúng

4. Cho đoạn mã C++ sau, hãy cho biết các đối tượng được cấp phát trong các vùng nhớ nào?

```
int * m;  
int * foo(int x) {  
    static int y;  
    int * z = new int;  
    switch x {  
        case 1: return &y;  
        case 2: return &x;  
        case 3: return z;  
        case 4: return m;  
    }  
}
```

Chọn:

- a. được cấp phát trong vùng nhớ heap (new, malloc, calloc)
- b. được cấp phát trong vùng nhớ stack
- c. được cấp phát trong vùng nhớ tĩnh

z	stack
new int	heap

5. Cho đoạn mã C++ sau

```
int* m;  
int* foo(int x) {  
    static int y;  
    int * z = new int;  
    switch x {  
        case 1: return &y;  
        case 2: return &x;  
        case 3: return z;  
        case 4: return m;  
    }  
}
```

Hãy điền tên của biến thích hợp vào các chỗ trống sau:

Chú ý:

- Chỉ điền tên 1 biến vào ô trống ngay cả khi có nhiều tên biến thích hợp để điền
- Nếu cần điền đối tượng được tạo ra bởi new int thì ghi là new int (7 ký tự bao gồm 1 khoảng trắng)

Đối tượng **x** có thời gian sống bằng với biến z.

Đối tượng **z** có thời gian sống bằng thời gian thực thi của hàm foo.

Đối tượng **y** có thời gian sống bằng với biến m.

Đối tượng **new int** có thời gian sống dài hơn thời gian thực thi của hàm foo nhưng không bằng thời gian thực thi của chương trình chính.

6. Cho đoạn chương trình sau viết trên một ngôn ngữ cấu trúc khối tựa Pascal:

```
var x,y:integer; // line 1

procedure a(x:real) //line 2

  var z: integer; // line 3

    procedure b(y:integer) // line 4

      var z: real; // line 5

      begin ... end //of b

    begin ... end // of a

begin ... end
```

Trong các câu sau, để đảm bảo so trùng, viết các tên tăng dần theo thứ tự alphabet và cách nhau bằng dấu phẩy, không có khoảng trắng. Chỉ kể các tên có trong đoạn chương trình trên, nếu có nhiều tên trùng nhau (như x,y,z) phải viết kèm dấu // và số line của khai báo, ví dụ x//1 hoặc y//4.

- Môi trường tham khảo cục bộ của thủ tục b gồm có các tên **y//4,z//5**
- Môi trường tham khảo không cục bộ của thủ tục b gồm có các tên **a//2,b//4,x//2**
- Môi trường tham khảo toàn cục của thủ tục b gồm có các tên **x//1,y//1**

→ Referencing Environment dạng này có quy tắc:

- Cục bộ (local):** Các biến bao gồm tham số trong () và { } (không bao gồm tên hàm)
- Không cục bộ (nonlocal):** những biến nằm trong hàm cha của nó.
- Toàn cục (global):** ngoài tất cả các produce.

7. Cho đoạn chương trình sau viết trên một ngôn ngữ cấu trúc khối tựa Pascal:

```
var x,y:integer; // line 1

procedure foo(x:real) //line 2

var z: integer; // line 3

begin .... end

procedure foo1(y:integer) // line 4

var z: real; // line 5

begin ... end

begin ... end
```

Giả sử chương trình chạy theo trình tự: **main gọi foo**, **foo gọi đệ qui foo**, **foo gọi foo1** với các đối tượng được cấp phát trên các bảng ghi hoạt động như sau:

	main	→ foo (1)	→ foo (2)	→ foo1
x	o1	x o3	x o5	y o7
y	o2	z o4	z o6	z o8

Hãy cho biết môi trường tham khảo động của foo1 có chứa các đối tượng nào?

- a. o3
- b. o4
- c. **o8**
- d. o2
- e. o1
- f. o6
- g. **o5**
- h. **o7**

8. Cho đoạn chương trình sau viết trên một ngôn ngữ tầm vực động:

```
Cho đoạn chương trình sau viết trên một ngôn ngữ tầm vực động:

program main;
```

```

var a,b,c,d: integer; // line 1

begin ... end

procedure sub0

  var c: integer;    // line 2

  begin .. end;

procedure sub1(a:real) //line 3

  var c:real;        //line 4

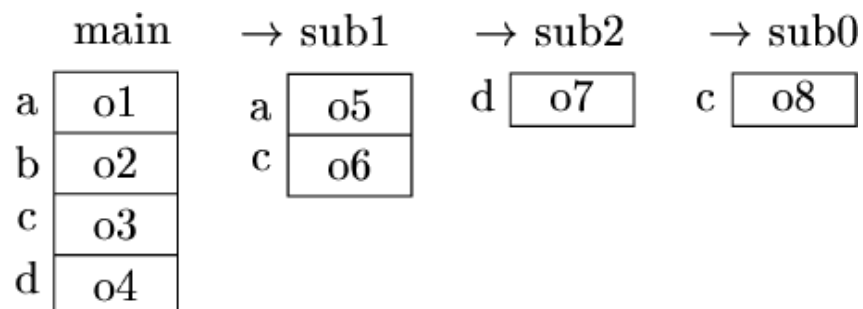
  begin ... end

procedure sub2(d:char) //line 5

  begin ... end

```

Giả sử chương trình chạy theo trình tự: main gọi sub1, sub1 gọi sub2, sub2 gọi sub0 với các đối tượng được cấp phát trên các bảng ghi hoạt động như sau:



Hãy cho biết môi trường tham khảo động của **sub0** có chứa các đối tượng nào?

- a. **o7**
- b. o6
- c. o4
- d. **o5**
- e. **o2**
- f. o1
- g. **o8**
- h. o3

9. Cho đoạn chương trình sau viết trên một ngôn ngữ tầm vực động:

```

program main;

var a,b,c,d: integer; // line 1

begin ... end

procedure sub0

var c: integer; // line 2

begin .. end;

procedure sub1(a:real) //line 3

var c:real; //line 4

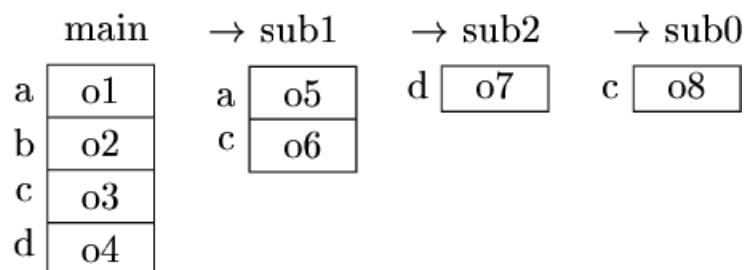
begin ... end

procedure sub2(d:char) //line 5

begin ... end

```

Giả sử chương trình chạy theo trình tự: main gọi sub1, sub1 gọi sub2, sub2 gọi sub0 với các đối tượng được cấp phát trên các bảng ghi hoạt động như sau:



Hãy cho biết môi trường tham khảo động của sub2 có chứa các đối tượng nào?

- a. **o6**
- b. **o7**
- c. o3
- d. o8
- e. o4
- f. **o2**
- g. o1
- h. **o5**

10. Cho đoạn chương trình sau viết trên một ngôn ngữ cấu trúc khối tựa Pascal:

```
var x,y:integer; // line 1

procedure foo(x:real) //line 2

var z: integer; // line 3

begin .... end

procedure foo1(y:integer) // line 4

var z: real; // line 5

begin ... end

begin ... end
```

Hãy cho biết các khai báo có hiệu lực trong procedure foo?

- a.
Khai báo x ở line 2
- b.
Khai báo foo1 ở line 4
- c.
Khai báo z ở line 3
- d.
Khai báo foo ở line 2
- e.
Khai báo y ở line 4
- f.
Khai báo y ở line 1
- g.
Khai báo x ở line 1
- h.
Khai báo z ở line 5

11.

Question**4**Not yet
answeredMarked out of
1.00

Flag question

Time left 0:06:37

Cho đoạn mã C++ sau, hãy cho biết khi gọi hàm foo với thông số là 3 thì sẽ có lỗi gì (trong các lỗi tham chiếu treo, garbage hoặc alias) xảy ra?

```
int * m;  
int * foo(int x) {  
    static int y;  
    int * z = new int;  
    switch x {  
        case 1: return &y;  
        case 2: return &x;  
        case 3: return z;  
        case 4: return m;  
    }  
}
```

Select one:

- ☐ a. Gây ra lỗi tạo ra rác
- ☐ b. Không gây ra lỗi gì cả
- ☐ c. Gây ra alias
- ☐ d. Gây ra lỗi tham chiếu treo (dangling reference)

B → Không gây ra lỗi gì cả.

- Nếu thứ được khai báo mà không trả ra (return) thì có rác.
 - Trong câu này, `return z` đã trả ra vùng nhớ int → không có garbage.
- Nếu trả ra một địa chỉ đã bị thu hồi (return & một biến đã thu hồi), thì lỗi tham chiếu treo.
 - Trong hình, nếu dùng case 2 thì bị lỗi tham chiếu treo và garbage.
 - Tham chiếu treo vì &x bị thu hồi nhưng lại trả ra &x.
 - Garbage vì new int không được trả về.

12.

Question**7**Not yet
answeredMarked out of
1.00

Flag question

Cho đoạn mã C++ sau:

```
int s = 0;  
int *p = &s;  
int *q = p;  
int r = *p;
```

Nhắc lại tác vụ &s trên C++ trả về địa chỉ của biến s, *p trả về giá trị nguyên của đối tượng mà p trỏ đến.

Hãy viết theo thứ tự alphabet các alias được tạo ra trong đoạn code trên?

Để đảm bảo trùng khớp, các tên phải được viết:

- cách nhau bằng dấu phẩy, không có khoảng trắng,
- theo thứ tự alphabet (ví dụ p,q)
- nếu có dấu *, & thì các dấu này viết trước ký tự (ví dụ *r,p), dấu * viết trước & (ví dụ *s,&r)

Answer:

Giải: Các tên đó là s, *p, *q

Alias cũng có thể là tên của chính nó, phải không nè :v

Đáp án: *p,*q,s

- p trở tới vùng nhớ của s
- q copy p nên p và q cùng trở về vùng nhớ của s
- r chỉ copy giá trị của p và mang một vùng nhớ khác